EgoQuant: Security and Compliance Report

For the judges of the 2026 Shenzhen FinTechathon, WeBank International Track, Topic B (Wealth Advisory Agent). Status: 20 October 2026. Every statement below names the file, the test or the measurement that backs it. Things that are not finished are listed in section 11 and are not claimed anywhere else. EgoQuant is a competition prototype, not a regulated service: no order is ever placed and no real money moves.

Evidence tags. `[code]` a file in the repository; `[test]` an automatic test that fails if the claim stops being true; `[live]` checked against the deployed system; `[measured]` a number we measured; `[design]` a deliberate decision with its reasoning in the named document. The repository is `fintechaton-backend` (Python / FastAPI) and `fintechaton-frontend` (Next.js); paths are relative to the backend unless stated.

---

1. The security design in ten sentences

1. **Decisions are code, explanations are AI.** Scores, categories, limits, weights, approvals and trade verdicts come from deterministic, tested code; a language model only explains results it is given, and one labelled exception (Agent D's news-tone label) never touches a number a user acts on [design: `CLAUDE.md`, `docs/AGENT_D_LIMITATIONS.md`].
2. **Suitability comes first.** A person's risk category is the **lower** of what their finances allow (capacity) and how they say and play they feel about risk (attitude: two thirds questionnaire, one third Risk Game); "I could not accept any loss" always gives the most cautious category [code: `egoquant/risk_profile/scoring.py`; `docs/RISK_GAME_BLEND.md`].
3. **An independent Risk Engine has the final word.** It recomputes every limit from the weights alone, without calling the optimiser, and only an approved portfolio is shown [code: `egoquant/risk_engine.py`].
4. **Behaviour can only tighten.** What a person actually did in past market falls (their trade history), a crash test and the game can lower a limit, never raise it [code: `egoquant/agents/behavioral/`].
5. **One person can never read another's data.** Every database call runs as the signed-in user under Row-Level Security, and the backend filters again; there is no all-powerful key in the system [code: `egoquant/accounts/store.py`; `supabase/migrations/`; test: `tests/accounts/`].
6. **Less data, kept shorter.** Broker credentials are never asked for, tokens are used once and dropped, uploaded files are parsed in memory, every stored row expires after 180 days, and "delete my data" removes everything it should (a test now checks that no table is forgotten) [test: `tests/accounts/test_migrations.py`].
7. **The AI is fenced in.** Its tools are whitelists with bounds, personal data is replaced by placeholders before any model call, every number in its answers must come from the engine, and a compliance filter withholds anything that reads like a trade instruction or a forecast [code: `chat.py`, `egoquant/advisor/`; test: `tests/test_chat*.py`, `tests/test_grounding_redaction.py`, `tests/accounts/test_chat_trade_tool.py`].
8. **Advice leaves a tamper-evident trace.** A confirmed, approved plan can be sealed: a salted 32-byte fingerprint of the full advice record is written to a public test blockchain, so any later change to a weight, an answer or a log line is detectable by anyone [code: `egoquant/anchoring/`; `docs/ANCHORING.md`; `docs/ON_CHAIN_EVIDENCE.md`].
9. **It fails safely and visibly.** When prices are slow it switches the whole build to a labelled snapshot; when the AI is down the engine still works; kill switches exist for every optional part; errors are plain and never carry a stack trace or a value the user sent [test: `tests/test_slow_feed_drill.py`, `tests/test_no_echo_422.py`, `tests/test_hardening_20oct.py`].

10. **We tell you what is not done.** Twenty-eight known gaps are listed with their owners (`docs/SECURITY.md` section 4), the category limits are labelled provisional until a bank confirms its mapping, and every report says that nothing in it is a forecast.

2. Scope and trust boundaries

```
Investor's browser ──HTTPS──> Next.js on Vercel ──HTTPS, Bearer token──> FastAPI on Railway ──> Supabase (Postgres + Auth, RLS)
  (no tokens: HttpOnly           (route handlers hold          (all decisions;                  ├─> OpenAI (explanations only, redacted)
   cookies only)                  the cookies; Origin check)    rate limits; body limits)       ├─> Tiingo / Binance / committed price snapshot
                                                                                                 ├─> IBKR Flex (read-only, one request)
                                                                                                 └─> Sepolia test chain (fingerprint only)
```

Zone	What it holds	Who can touch it
Browser	The pages and the person's own answers in local storage (a signed-in person's copy is also kept in their account). **No token**: sessions live in `HttpOnly`, `Secure`, `SameSite=Lax` cookies set by the server.	The person
Vercel (frontend)	Route handlers that turn the cookie into a Bearer header, check the request's `Origin`, rate-limit logins, and forward to the backend	Deployment owner (Savas)
Railway (backend)	All scoring, optimisation, risk checks, trade checks, the advisor logic. Stores nothing itself except a price cache.	Deployment owner
Supabase	Accounts, saved reports, imported trades, consents, logs, onboarding state. Row-Level Security on every table.	Each user their own rows; the project owner (Bideci) the settings
OpenAI	Receives only engine facts and aggregates with personal values replaced by placeholders (section 7). `store=false`.	Provider
Public blockchain	Only `EGOQ1` + a salted SHA-256 fingerprint (section 8)	Everyone (read)

Out of scope and said so: the hosting providers' own security, a licensed market-data feed (we use a committed snapshot plus free feeds), and real brokerage execution (there is none).

3. Suitability and investor protection (the compliance core)

The questionnaire, its scoring and its limits are in `egoquant/risk_profile/` (README with the reason for every cut-off).

Principle	How EgoQuant does it	Evidence
Know the client	A 25-question questionnaire (26 steps on the counter, one question asks for two amounts): the **Grable and Lytton** 13-item risk-attitude scale verbatim (score 13-47 in five levels), a **capacity** section (horizon, money needed back, emergency fund, share of wealth, income stability), a **knowledge** check (the **Lusardi and Mitchell** "Big Three" questions, verbatim) and a goal.	`[code] risk_profile/questions.json`, `README.md`; `[test] tests/risk_profile/` (134 tests)
Attitude and capacity are different things	Category = the **lower** of capacity and attitude. Capacity is the average of its answers, with "money you must get back" a hard cap and a horizon under a year or one to three years a hard cap too (2 and 3).	`[test] tests/risk_profile/test_risk_profile.py`

Principle	How EgoQuant does it	Evidence
Behaviour is evidence, not decoration	The **Risk Game** (about two minutes, offered to everyone) counts one third of the attitude to risk; the category can never pass the capacity cap; "no loss at all" forces the lowest category. A person's real trade history, a crash test and the game can **only tighten** the portfolio's loss limit.	`[test]` `tests/test_risk_game_routes.py`, `tests/system/test_risk_game_blend.py` (95 tests, 300-investor sweep), `tests/agent_b/` (680 tests)
Product risk matches client risk	Every product has a risk level R1-R5; a client in category Cn may hold products up to the level the category allows, plus per-class caps (stocks, gold and commodities, crypto, cash) and a loss limit in a named hypothetical stress scenario. Complex products (crypto, leveraged funds) need passing the knowledge check.	`[code]` `egoquant/models.py` (class policies), `risk_profile/config.yaml`, `egoquant/agents/market_intel/catalog.py`; `[test]` `tests/risk_profile/`, `tests/agent_c/`
Independent verification	The Risk Engine re-checks R-level, instrument caps, stock share, class caps and stress loss; an unknown product or a missing limit **fails closed**. A failed check is shown to the person in plain words.	`[test]` `tests/system/`, `tests/test_plain_wording.py`
Recorded and explainable	The report states how the category was set (`category_basis`), every limit with its source, the reasons in fixed templates, and the stress results; the questionnaire answers are audit-recorded with a version and a hash; a profile has a validity date and an expired profile is flagged, not silently used.	`[code]` `egoquant/report.py`, `risk_profile/scoring.py` (`audit_records`)
Honest disclosures	Every number is labelled hypothetical or in-sample; "N in 100 simulated futures" instead of probabilities; FINRA Rule 2214-style disclosures; a banner marks the category limits as provisional ("DRAFT LIMITS") until a bank confirms its own mapping.	`[test]` `tests/system/test_report.py`; `[design]` `docs/DEMO_SCRIPT.md` section "Is this regulated advice?"
Advice, not execution	No order can be placed; the broker link is read-only; the UI and the API never ask for a broker username or password.	`[code]` there is no order code; `[test]` `tests/accounts/`

Vocabulary note. The C1-C5 customer categories and R1-R5 product levels follow the grading customary in Chinese wealth-management practice; the numeric limits behind them are our placeholders (marked PROVISIONAL in the code and shown as a banner), to be replaced by an institution's own table. We make no claim of certification under any rulebook; the controls are designed so that the mapping (tolerance versus capacity versus knowledge, suitability of product to client, disclosure, record-keeping, data-subject rights) is straightforward.

4. Identity, sessions and access control

Threat	Control	Evidence
Account takeover, weak passwords, brute force	Supabase Auth (hashing, e-mail confirmation, minimum password length 10, only the e-mail provider enabled, anonymous sign-ins off); login and sign-up are rate-limited in the frontend handlers; no password reaches our backend.	`[live]` settings recorded in `docs/supabase/SUPABASE_LOG.md`; `[code]` `fintechaton-frontend/src/lib/server-auth.ts`
Session theft	Tokens only in `HttpOnly`, `Secure`, `SameSite=Lax` cookies (`__Host-` prefix in production); the page never holds a token.	`[code]` `src/app/api/auth/*`

Threat	Control	Evidence
CSRF	State-changing `/api/account/*` and `/api/auth/*` calls check the request's `Origin`; the backend accepts Bearer tokens, not cookies.	`[code]` `sameOrigin()`
IDOR (user A reads user B)	No endpoint takes a user id: identity comes only from the verified token. Every query runs **as the user** so Supabase RLS applies, **and** carries the backend's own `user_id` filter. There is no service-role key anywhere; the account store **refuses to start** with a privileged key.	`[test]` `tests/accounts/` (265 tests: A versus B on every endpoint, smuggled `user_id`, forged tokens); `supabase/tests/rls_isolation.sql` run on the real project (**PASS**, 19 Oct, `docs/supabase/SUPABASE_LOG.md`)
Direct database access with the public key	Table-level and column-level grants (no `UPDATE` / `DELETE` where none is needed, inserts cannot set `created_at` or `expires_at`), RLS "own rows", verified with the Data API without a token (401 / permission denied).	`[code]` `supabase/migrations/001`, `002_hardening`, `005_what_if_hardening`; `[live]` probe recorded in the log
Garbage or expired token	A token the auth service cannot read is a plain 401, never a 5xx.	`[test]` `tests/test_hardening_20oct.py`

5. Input handling, abuse resistance and application hygiene

Topic	Control	Evidence
Validation	Every request body is a strict schema (`extra="forbid"`, ranges, lengths); unknown fields are rejected without naming them.	`[test]` `tests/test_no_echo_422.py`, `tests/test_validation_messages.py`
No echo	A 422 names the field in words and the rule ("Horizon in years: must be above 0") and never repeats what was sent, so a credential pasted into a wrong field cannot come back.	`[test]` thirteen canary values on the **live** server never came back (Damla's probe, 20 Oct, `docs/judges/04_TESTING_AND_EVIDENCE.md`)
Hostile input	80 hostile-input cases on account and trade-check routes (oversized bodies, unicode and null bytes, NaN/Infinity, forged tokens, path-like ids, formula cells), 33 more on the Risk Game field, 129 extreme-but-valid investor inputs: no 5xx, no NaN in any number.	`[test]` `tests/accounts/test_extreme_inputs.py`, `tests/system/test_risk_game_blend.py`, `tests/system/test_engine_edge_cases.py`
Uploads (IBKR reports)	`defusedxml` only and an early refusal of DOCTYPE/ENTITY; 2 MB and 20,000-row caps; extension must match content; ZIP/PDF/XLS refused; symbols and currencies must match strict patterns (so formula-like text never gets through); parsed in memory, never written to disk.	`[test]` over 90 tests incl. XXE, billion laughs, formula cells (`tests/agent_b/`)
Size and rate limits	Body size capped by declared length (4.5 MB / 512 KB / 1 MB, 413 or 411); portfolio and trade-check 20 per minute; chat 120 per minute; accounts 30 per minute per user; what-if 10 per 10 minutes per user; chat trade checks 10 per 10 minutes per user; the 429 answers carry `Retry-After`.	`[test]` `tests/test_body_limit.py`, `tests/test_hardening_20oct.py`

Topic	Control	Evidence
Headers and CORS	`nosniff`, frame denial, strict referrer policy, no camera/microphone/geolocation, HSTS; CORS allows only the Vercel origin and no credentials; the frontend sets a content-security policy.	`[live]` `python eval/deploy_check.py <backend>` (security headers on 200, 404, 405, 413, 422)
Redirects	`redirect_slashes` is off and the proxy headers are trusted, so a request is never redirected to `http://`.	`[test]` `tests/test_hardening_20oct.py`
Errors	One generic 500 with no stack trace or exception text; the cause is logged by type and route only.	`[code]` `main.py`
Debug surface	`/debug/portfolio` answers 404 for any body unless `EGOQUANT_DEBUG_PAGES` is on (off in production); `/docs` and `/data/status` are public by design (they contain no secret).	`[test]` `tests/test_debug_gate.py`, `tests/test_hardening_20oct.py`
Secrets and dependencies	Keys only in the host environment or a git-ignored `.env`; staged diffs are secret-scanned before every push; `requirements.txt` pins every package (the numerical libraries since 20 Oct, to the versions we tested; `/data/status` shows the running versions); a dependency audit (`pip-audit -r requirements.txt`, 20 Oct, 15:12) found no known vulnerability in the 56 pinned packages; the app never calls Yahoo (a test checks `yfinance` is not a dependency).	`[test]` `tests/test_hygiene.py`
Concurrency	Eight different investors built at once give the same results as the same investors built alone, on the same machine and library versions (no shared state leaks); memory stays flat over 30 builds. A gate on simultaneous heavy builds (default 1, `EGOQUANT_MAX_CONCURRENT_BUILDS`) protects a small server.	`[test]` `tests/system/test_concurrent_builds.py`; `[measured]` 8 simultaneous builds peak at 357 MB and stay flat; 30 sequential builds stay at 235-247 MB

6. Privacy and data protection

What we hold and for how long (all rows expire after 180 days; a daily `purge_expired()` job deletes them when `pg_cron` is on):

Data	Where	Why	Who can read	Removed by
E-mail, password hash, chosen name	Supabase Auth	sign-in	the person (Auth)	"delete my account" (cascade)
User name, picture (preset or a small checked JPEG/PNG/WebP, never SVG)	`profiles`	display	the person	account deletion (the picture also by "delete my data")
Questionnaire answers, preferences, goal inputs, Risk Game result, built-report pointer	`onboarding` (one row per user)	continue on another device	the person	"delete my data"
Saved reports, with the answers they were built from	`reports`	show and seal them	the person	"delete my data"

Data	Where	Why	Who can read	Removed by
Imported trades and cash flows (date, symbol, side, quantity, price, currency, category, net cash only)	`trades`, `cash_flows`	holdings and behaviour	the person	"delete my data"
Consents, operation log (counts, dates, labels), trade checks, what-if runs, privacy counts	their own tables	proof of consent, audit	the person	"delete my data"
Sealed advice records and seals	`advisory_records`, `record_seals`	tamper-evidence	the person	"delete my data" (the on-chain fingerprint stays, but we can no longer link it to anyone: section 8)

Never stored: broker username or password (never asked for), IBKR Flex tokens (used for one request, then dropped), uploaded files (parsed in memory), chat text, price-feed keys in the repository. **Consent:** a versioned consent is required before broker data is connected, another for trade checks, another before sealing; a changed text needs new consent. **Rights:** the person can see their data (Account page), export a report, delete all data (`DELETE /account/data`) and delete the account (`DELETE /account`). Section 11 lists the one place (the profile row and picture) where "delete my data" intentionally keeps the account itself. **Erasure is complete by construction:** three migrations used to redefine the erase function with only their own tables; `007_erase_everything.sql` is now the single final version and `tests/accounts/test_migrations.py` fails if any table with a `user_id` or an expiry is missing from it [test]. **Third parties (processors):** Vercel and Railway (hosting), Supabase (accounts and data), OpenAI (explanations, redacted), Tiingo/Binance (prices; no personal data is sent), a public RPC (fingerprints only). **Data minimisation toward the AI:** section 7.

7. AI governance (the part judges will ask about most)

Risk	Control	Evidence
The AI makes a decision	It has no write path to any score, category, limit, weight, approval or verdict. Its only tools are `run_what_if` (re-runs the deterministic engine on a validated scenario) and `check_trade` (runs the deterministic trade check; signed-in investors only). Arguments are strict whitelists with no field for limits, categories or identities; at most two tool calls per message.	`[test]` `tests/test_chat_what_if.py`, `tests/accounts/test_chat_trade_tool.py` (38), `tests/system/test_what_if.py`
It invents a number	**Number grounding**: every percentage, "X in 100", money amount and multiplier must come from the engine facts, this turn's tool results or the user's own words; otherwise the reply is regenerated once with a correction, then replaced by a safe fallback.	`[test]` `tests/test_grounding_redaction.py`
It gives trade signals or forecasts	A **compliance guard** withholds any sentence that reads like "you should buy", a price target, a guarantee or a forecast; the prompt tells it to report what the *check found*, not what to do.	`[code]` `egoquant/advisor/guard.py`, `config.yaml` (36 patterns)
It leaks personal data	A **redaction gateway** replaces account ids, e-mails, phone numbers and the user's name with placeholders on the way out and restores them in the reply; it only logs counts. The investor's first name, used for a greeting, reaches the model only as a token (`[NAME_1]`). The model never receives balances, holdings lists or raw trades; trade questions send a verdict and the reasons.	`[test]` `tests/test_grounding_redaction.py`, `tests/accounts/test_chat_trade_tool.py`

Risk	Control	Evidence
Prompt injection	Conversation history and news headlines are treated as untrusted data; sealed facts are HMAC-signed, so another kind of token (news, what-if) is never accepted as chat facts; injection-shaped tool arguments ("use C5 limits") are rejected by the whitelist.	`[test]` `tests/test_chat_what_if.py`; `[measured]` the live runs kept in `eval/results/` (advisor chat, prompts v2 to v5: 19/19, 19/19, 21/21 and 24/24; trade explanation: 22/24 then 24/24, 24/24 and 51/51 twice)
The AI is down or out of credit	The report, limits, trade checks and what-if numbers do not depend on it; the chat answers one plain sentence; explanations fall back to templates.	`[test]` `tests/advisor/test_ai_down.py`, `tests/test_chat.py`
Silent change of behaviour	The prompt has a version (`advisor-v11`) written to the audit log; changing it requires the live red-team. **The live red-team of 20 Oct found a real defect in advisor-v10**: the model filled both size fields of the trade tool on every call, the server refused the call, and no trade check completed (a safe failure: no advice, no leak). The tool now takes one size (advisor-v11), and the full 42-case run on v11 passed **41 of 42** (all 24 non-trade cases, 17 of 18 trade cases; the one miss is a false positive of the test's keyword check on a correct reply) (section 10).	`[code]` `chat.py` `PROMPT_VERSION`; `eval/chat_redteam.py`; `[measured]` `eval/results/chat_redteam_20261020_164237.jsonl`
The one AI exception	Agent D rates each headline's tone **for one named fund** with an LLM (SHA-256-pinned prompt, cache, FinBERT fallback). The label feeds only the news tab's tone summary and an alert that *offers* a profile re-check; it never sets a number the person acts on. Its measured agreement with team labels was 85% (FinBERT: 53%).	`[design]` `docs/AGENT_D_LIMITATIONS.md`, `egoquant/briefing/README.md`

8. Tamper-evidence: the on-chain seal

In plain words: when you confirm a plan, EgoQuant takes a digital fingerprint of the exact final report (rendered once when you confirm) and publishes only that fingerprint on a public test blockchain. Anyone can later check a report against it; a report that was changed does not match.

In detail: when a signed-in person confirms an *approved* plan (once the news rating of the report is finished), EgoQuant renders the final report **once** into a *sealed document* whose header says "Sealed for: <username> · report <id>", and stores it byte for byte. It builds a record in a fixed format (category, answers, the portfolio weights to the millionth, the Risk Engine's checks, code and model versions, the steps the person took, the SHA-256 of the sealed document and the person's chosen username), adds a random secret number (a salt), and publishes the fingerprint `SHA-256(salt || record)` on the Sepolia **test** network in a transaction that moves no money (0 ETH), from EgoQuant's sealing address to itself, labelled `EGOQ1`. The person downloads ONE sealed report (the document followed by its seal); the **Verify a report** page (no account needed) recomputes both fingerprints in the browser, reads the chain, and answers "Unchanged ✓ · Sealed for: <username> · confirmed <date> · block N" or "Does not match ✗: this report was changed after it was sealed". - **What it proves:** this exact report existed, unchanged, no later than that block's time, sealed for that username. It closes one gap that a simpler seal would leave (a genuine seal shown next to an *edited* report) and narrows another (someone else's genuine report: the report itself says whom it was sealed for, but the Verify page does not yet compare that with the viewer's account). **What it does not:** that the advice was right, or who is holding the file; a printed or PDF copy cannot prove itself (only the sealed .html or the seal file can be checked), and "this report belongs to my account" is shown on **Account, My seals** but not yet checked automatically on the Verify page (decision OC-15 in `docs/ONCHAIN_DECISIONS.md`). - **Privacy:** a public chain entry cannot be deleted. Without the record and its salt the fingerprint reveals nothing about the record and cannot be tested against guesses (the chain does

show that a seal was made, when, and from which address); "delete my data" deletes the record, the salt and the sealed document (one database row), so we can no longer link the fingerprint to a person from our database (the database provider's backups age out under its own rules; someone who kept a downloaded sealed report still holds its record and salt). The record carries the username the person chose (it can be a real name; never an e-mail or a user id) inside the salted fingerprint only: the chain sees only the label EGOQ1 and the 32-byte fingerprint. The consent text says this before the person agrees. - **Limits of a test network:** Sepolia keeps its entries only as long as the test network runs and makes no promise to keep them forever (a test network can be reset or retired). A lasting record would be a property of the chain a real deployment would use (a consortium chain, below; not built), not of Sepolia. **A check that does not use our website:** the Verify page runs code that our own website serves, so a compromised website could print "Unchanged"; the same checks run without our website with the command-line verifier `python -m egoquant.anchoring verify <file>` (a sealed report or a seal file; it is our own code and reads the chain through a provider; `tests/anchoring/test_proof_cli.py`). - **Safety:** testnet only (any other chain id is refused), the key is read only from the service's environment, it is not in the repository and no public endpoint returns it (`tests/test_no_secrets_in_public_endpoints.py`), sealing never blocks or changes advice (when the chain is down the plan is confirmed and the seal waits), 5 seals per user per day. Seals made before this version still verify ("older seal format, no document fingerprint"). - **Evidence:** five real seals of real engine output for invented investors with their Etherscan links and tampered-copy results are in `docs/ON_CHAIN_EVIDENCE.md`; the Verify page's own demo report is a real sealed report of an invented investor (block 11,951,549); every committed proof still verifies and every tampered copy still fails (`tests/anchoring/test_committed_proofs.py`); 465 tests in `tests/anchoring/` (464 run by default; one live test needs the key) (`test_seal_v2.py`: one changed character anywhere, a changed username, a forged footer, older seals, My seals isolation, no-migration fallback) and the frontend repository's vector check (323 checks of the browser code against a report made by the backend); the decisions are recorded in `docs/ONCHAIN_DECISIONS.md`; a click-by-click path for judges is `docs/JUDGE_GUIDE.md`. - **Roadmap:** the same fingerprint format would go to a consortium chain such as WeBank's FISCO BCOS.

9. Availability and resilience

Failure	What happens	Evidence
Live price feed slow (about 15 s per live request, estimated from a 195 s build of 20 interest areas on the host, 19 Oct)	After 15 s of waiting, or two slow requests, the **whole** build switches to the committed snapshot of real prices and says so; `EGOQUANT_PRICE_MODE=snapshot` sends no live request at all. A build with 20 interest areas used to take 195 s on the host; in snapshot mode it takes seconds.	`[test] tests/test_slow_feed_drill.py` (5 scenarios), `tests/test_portfolio_time_budget.py`; `[live] /data/status`
News/AI out of time	The in-request briefing is capped at 5 s; the report is sent without it and the briefing job fills the tab.	`[code] egoquant_routes.py`
Heavy simultaneous use	Rate limits per route, and a **build gate**: one heavy build at a time by default (`EGOQUANT_MAX_CONCURRENT_BUILDS`), the others wait their turn; through the website the wait ends after 115 s with the website's own "too slow" answer, and a direct API call gets, after 120 s, a plain 503 with `Retry-After`. **Measured locally (20 Oct, Damla):** 20 of the heaviest (60-year) builds in the same second reached about 3.2 GB and killed the server process in 3 of 4 runs without the gate; with one slot nothing crashed (about 300 MB) and 8 to 11 of the 20 were served, the rest turned away; ordinary builds were not flood-tested. One math-library thread per process is the default (`main.py`).	`[measured]` flood table in `docs/judges/04_TESTING_AND_EVIDENCE.md`; `[test] tests/test_build_gate.py` (7), `tests/test_stack_safety.py`

Failure	What happens	Evidence
A subsystem misbehaves	Kill switches (exact table in `docs/DEPLOY_ENV.md`): the price builder, live Tiingo, the snapshot fallback, the regime sampler (`EGOQUANT_REGIME_MC=0`), news (`EGOQUANT_BRIEFING=off`), the news worker, the chat (unset the key), accounts (unset Supabase), the Risk Game (`EGOQUANT_RISK_GAME_WEIGHT=0`), portfolio method v2 (off by default).	`[code]` `egoquant/settings.py`
Wrong deploy	A read-only `python eval/deploy_check.py <backend>` checks status, Python version, price mode, headers, CORS, the Risk Game on three investors, builds against the demo pack, trade checks and the what-if refusal without a token; it refuses to build on a server that would send live price requests.	`[test]` `tests/test_deploy_check.py` (31); `[live]` 20 Oct 13:00, after the price mode was set to snapshot: 71 of 71 checks passed (before that 25 passed, 1 failed, which was the price mode, and 1 was a warning: the builds were skipped on purpose)

10. How we verified (the evidence in numbers)

Evidence	Result
Automatic tests (offline, no network)	The count, the areas and the last complete runs are in `docs/judges/04_TESTING_AND_EVIDENCE.md` section 1 (kept in one place so that they cannot disagree). The last complete runs passed except one known timing test of the news page (`tests/briefing/test_news_ui.py`), which does not touch an investment rule.
Mutation checks	Hostile-test authors broke the code on purpose (wrong blend, floor instead of round, no cap, gate not forcing the lowest category, a forgotten table in erasure, a record that drops the Risk Game result): every mutation was caught.
Property sweeps	300 random investors through the real endpoints for the category rule (`tests/system/test_risk_game_blend.py`); 5,000 random trades for the trade check (`tests/system/test_trade_check_sweep.py`); 100 end-to-end builds.
Live red-teams (OpenAI, 18 and 19 Oct, earlier prompts; raw results in `eval/results/`)	Advisor chat, prompts v2 to v5: 19/19, 19/19, 21/21 and 24/24 (`chat_redteam_*.jsonl`). Trade explanation: v1 22/24 (both failures one Chinese-language case, fixed in v2 by "English only"), v2 24/24, v3 24/24 and two further v3 runs of 51/51 (`trade_explain_redteam_*.jsonl`). On 20 Oct, live model, current prompts: trade explanation v4 **51/51** (17 cases x 3 verdicts; 50 replies written by the model, 1 by the template; `trade_explain_redteam_20261020_162406.jsonl`); advisor chat v10 **34/42** (all 24 non-trade cases passed; 8 of 18 trade cases failed safely for one cause: the model filled both size fields of the trade tool, so the server refused the call and no verdict was given; a C4 variant gave the same 34/42; `chat_redteam_20261020_161517.jsonl`, `_163303.jsonl`); the tool was changed to one size (**advisor-v11**) and re-run: the 18 trade cases **17/18** (`_163812.jsonl`) and the full 42 cases **41/42** (`_164237.jsonl`; the one miss is a false positive of the keyword check on a correct reply). A regime-wording check run on 19 Oct was not kept.
Real-project proof	`supabase/tests/rls_isolation.sql` passed on the real Supabase project; the Data API refuses anonymous reads on every table.
Live hostile probe	93 read-only requests against the previous deploy and 18 more after the 12:38 deploy (18 of 18 answered as expected): 13 canary values never echoed, all error statuses carry the security headers, CORS only for the Vercel origin, protected routes answer 401 plainly (`docs/judges/04_TESTING_AND_EVIDENCE.md`).
Fourteen-investor reading	Fourteen invented investors, 30 builds, 26 trade checks, 14 what-ifs and 31 wrong requests: the category rule held for every one; 19 wording findings fixed (`docs/judges/03_METHODS_AND_MODELS.md` section 6).

Re-run it yourself. Backend tests: `PYTHONUTF8=1 .venv/Scripts/python -m pytest -q -m "not live"`. Deployed system: `python eval/deploy_check.py https://web-production-ee5cc.up.railway.app`. On-chain: `python -m egoquant.anchoring verify docs/anchoring/real-balanced_saver.proof.json`. Threat table with owners: `docs/SECURITY.md`.

11. Residual risks and honest gaps (what we would do next)

1. **Category limits are placeholders** until a bank confirms its mapping (banner on every report).
2. **Client-reported inputs.** The questionnaire and the Risk Game are self-reported; a person can replay the game until a wanted category appears. The effect is bounded (one third of the attitude, never past the capacity cap, plus an optional switch so the game may only lower the category). A saved "result of record" rule is a roadmap item.
3. **Account features were proven at table level on the real Supabase project** (both RLS proofs pass, the Data API refuses anonymous reads on all 12 tables, migrations 001 to 008 applied; `docs/supabase/SUPABASE_LOG.md`). A signed-in walk of the seal path on the public site was done by a team member on 20 Oct (build, confirm and seal, download, verify, tamper: Sepolia block 11,954,258); the two-user smoke test through the deployed backend (`docs/supabase/RUN_LIVE_SMOKE.md`) is still to be recorded.
4. **E-mail confirmation uses Supabase's small shared sender**; judges need a working mailbox or a demo account.
5. **The live red-team of 20 Oct found one real defect in the advisor (advisor-v10)**: the trade tool's two size fields, which the model always both filled, so no trade check completed (a safe failure). It was fixed the same hour (advisor-v11, one size field) and re-run: 41 of 42 cases pass (the one miss is a false positive of the test's own keyword check); the trade explanation (trade-explain-v4) passed 51 of 51. The red-team is a 42-case suite of our own, not an external audit.
6. **A public blockchain entry cannot be deleted** (mitigated by the salt, section 8). Sepolia is a test network.
7. **Rate limits are per process**, not per IP (requests arrive through the frontend proxy); the heavy-build cap is also per process.
8. **The frontend CSP allows inline scripts** (a Next.js requirement); there is no nonce-based policy yet.
9. `operation_log` **is an activity record**, not a tamper-proof audit log (a user can add entries for themselves).
10. **Yahoo Finance data in the committed snapshot** is accepted for the competition only; a commercial product needs a licensed feed.
11. **Binance and further broker imports, Chinese-language advice, and a consortium chain** are roadmap, not built.
12. **The old Tiingo key is in git history** and must be rotated.

Source documents: `docs/SECURITY.md` *(threat table T1-T19 and the full gap list),* `docs/RISK_GAME_BLEND.md`, `docs/ANCHORING.md`, `docs/DEPLOY_ENV.md`, `egoquant/risk_profile/README.md`, `egoquant/briefing/README.md`.